Requirements Document

Introduction

This document defines the requirements for migrating the HydUrban application from a JSON file-based storage system to a PostgreSQL database, restructuring the dual-backend architecture so the .NET API becomes the single data API, and adding a complete user authentication and personalization system (saved properties, favorites, saved searches, and comparison results).

The migration is executed in phased steps to avoid breaking changes, with the Python Flask service narrowing to scraper orchestration only and all frontend data access going through the .NET API.

Glossary

- **System**: The HydUrban application (all backend and frontend components collectively)
- **API**: The .NET Web API service running on port 5001
- **Flask_Service**: The Python Flask service running on port 5000, responsible for scraper orchestration
- **Scraper**: A Python script that retrieves RERA, SRO, or RR data from external sources
- **PostgreSQL**: The primary relational database introduced by this migration
- **Repository**: A .NET class implementing the data access layer for a given domain entity
- **JWT**: JSON Web Token used for stateless authentication
- **Access_Token**: A short-lived JWT (15 minutes) identifying the authenticated user
- **Refresh_Token**: A long-lived opaque token (30 days) used to obtain new access tokens
- **User**: A registered end-user of the application
- **Admin**: A user with the `admin` role, able to trigger scrapes and manage content
- **Saved_Property**: A deliberate bookmark a user places on a project for later review
- **Favorite**: A quick heart-button action a user takes on a project
- **Saved_Search**: A named set of search filter criteria persisted for reuse
- **Comparison_Result**: A persisted side-by-side comparison of two or more projects
- **BCrypt**: The password hashing algorithm used with cost factor 12
- **Dapper**: The micro-ORM used by the .NET API for PostgreSQL queries

- **Npgsql**: The .NET PostgreSQL driver

- **psycopg2**: The Python PostgreSQL driver used by scrapers

- **JSONB**: PostgreSQL's binary JSON column type used for flexible schema fields

- **Feature_Flag**: A configuration value (`UsePostgresRepository`) that toggles the active repository implementation

- **Email_Service**: The SMTP or SendGrid integration used to send transactional emails

- **Input_Sanitizer**: The existing `IInputSanitizer` service applied to all user-provided strings

---

Requirements

Requirement 1: PostgreSQL Database Provisioning

User Story: As a system operator, I want a PostgreSQL database with a well-defined schema, so that project data, user data, and analytics can be stored, queried, and indexed reliably.

Acceptance Criteria

1. THE System SHALL provision a PostgreSQL database named `hydurban` accessible to both the Flask_Service and the API.

2. THE System SHALL create the `projects` table with columns: `id`, `project_name`, `project_status`, `project_type`, `district`, `mandal`, `locality`, `pin_code`, `village`, `approved_date`, `completion_date`, `revised_completion_date`, `total_area_sqmt`, `net_area_sqmt`, `built_up_area_sqmt`, `mortgage_area_sqmt`, `promoter_name`, `org_type`, `bank_name`, `branch_name`, `plan_approval_number`, `survey_number`, `is_msb`, `has_litigation`, `total_flats`, `total_booked`, `saleable_area_sqmt`, `raw_data`, `pricing`, `available_documents`, `scraped_at`, `updated_at`.

3. THE System SHALL create the `sro_transactions` table with columns: `id`, `sro_name`, `district`, `village`, `apartment`, `flat_no`, `reg_date`, `quarter`, `mkt_value`, `cons_value`, `price_per_sqft`, `scraped_at`.

4. THE System SHALL create the `unit_rates` table with columns: `id`, `district`, `mandal`, `locality`, `search_type`, `unit_rate_sqft`, `scraped_at`.

5. THE System SHALL create the `leads` table with columns: `id`, `name`, `email`, `mobile`, `area_of_interest`, `project_name`, `project_id`, `device_fingerprint`, `source`, `created_at`.

6. THE System SHALL create the `scrape_runs` table with columns: `id`, `scraper`, `status`, `started_at`, `finished_at`, `total`, `completed`, `errors`.

7. THE System SHALL create indexes on `projects(pin_code)`, `projects(district)`, `projects(project_status)`, `projects(locality)`, and a GIN index on `projects(raw_data)`.

8. THE System SHALL create indexes on `sro_transactions(village)`, `sro_transactions(quarter)`, `sro_transactions(apartment)`.

9. THE System SHALL create indexes on `unit_rates(mandal)` and `unit_rates(district)`.

10. THE `projects.raw_data` column SHALL be declared as type `JSONB` and SHALL NOT be nullable.

11. THE `leads.project_id` column SHALL reference `projects(id)` as a foreign key.

Requirement 2: Scraper Database Integration (Python / psycopg2)

User Story: As a system operator, I want the Python scrapers to write data directly to PostgreSQL, so that scraped project and analytics data is immediately available for querying without intermediate JSON files.

Acceptance Criteria

1. WHEN the RERA scraper completes processing a project, THE Flask_Service SHALL insert or update a row in the `projects` table using an `INSERT ... ON CONFLICT (id) DO UPDATE` statement that sets `raw_data` and `updated_at`.

2. WHEN the SRO scraper completes processing a transaction batch, THE Flask_Service SHALL insert rows into `sro_transactions` using `INSERT ... ON CONFLICT DO NOTHING` to ensure idempotency.

3. WHEN the RR scraper completes a run, THE Flask_Service SHALL truncate `unit_rates` and re-insert the new dataset.

4. THE Flask_Service SHALL use parameterized queries via `psycopg2` for all database write operations.

5. THE Flask_Service SHALL read the database connection string from a configuration source (`.env` or `scrape_preferences.json`) rather than embedding credentials in source code.

6. THE Flask_Service SHALL install `psycopg2-binary==2.9.10` as a dependency.

7. WHEN a scraper run starts, THE Flask_Service SHALL insert a row into `scrape_runs` with `status = 'running'`.

8. WHEN a scraper run finishes successfully, THE Flask_Service SHALL update the corresponding `scrape_runs` row with `status = 'completed'`, `finished_at`, `total`, and `completed` counts.

9. IF a scraper run encounters a fatal error, THEN THE Flask_Service SHALL update the corresponding `scrape_runs` row with `status = 'failed'` and append the error details to the `errors` JSONB array.

Requirement 3: Phased Migration with Feature Flag

User Story: As a developer, I want a feature flag that toggles between the file-based and PostgreSQL repository implementations, so that I can verify data parity before fully switching over without breaking the running application.

Acceptance Criteria

1. THE API SHALL read a boolean configuration value `FeatureFlags:UsePostgresRepository` from `appsettings.json`.

2. WHEN `UsePostgresRepository` is `false`, THE API SHALL use the file-based `ProjectRepository` implementation.

3. WHEN `UsePostgresRepository` is `true`, THE API SHALL use the `PostgresProjectRepository` implementation.

4. THE System SHALL support Phase 1 operation where scrapers write to both JSON files and PostgreSQL simultaneously.

5. WHEN Phase 2 is activated by setting `UsePostgresRepository` to `true`, THE API SHALL serve all project data exclusively from PostgreSQL without requiring a restart of the scraper service.

6. THE API connection string SHALL be stored in `ConnectionStrings:DefaultConnection` within `appsettings.json` and SHALL NOT be hardcoded in source files.

Requirement 4: .NET PostgreSQL Repository

User Story: As a developer, I want a PostgreSQL-backed project repository in the .NET API, so that the API can query, filter, and return project data efficiently using SQL instead of scanning the filesystem.

Acceptance Criteria

1. THE API SHALL include a `PostgresProjectRepository` class that implements the existing `IProjectRepository` interface.

2. THE `PostgresProjectRepository` SHALL use `Npgsql` version `8.0.5` and `Dapper` version `2.1.35` for all database interactions.

3. THE API SHALL expose a `GetAllProjectsAsync()` method that returns all projects ordered by `project_name`, selecting `id`, `project_name`, `project_status`, `project_type`, `district`, `mandal`, `locality`, `pin_code`, `total_flats`, `total_booked`, `saleable_area_sqmt`, `raw_data`, `pricing`, `available_documents`, and `scraped_at`.

4. THE API SHALL expose a `GetProjectByIdAsync(string id)` method that returns the full project row for the given `id`, or `null` if not found.

5. THE API SHALL expose a `GetProjectsByPinCodeAsync(string pinCode)` method that returns only rows where `pin_code` matches the supplied value.

6. THE `PostgresProjectRepository` SHALL use parameterized queries for all SQL statements to prevent SQL injection.

7. WHEN a database connection cannot be established, THE API SHALL return a `503 Service Unavailable` response with a descriptive error message.

Requirement 5: Architecture Simplification

User Story: As a system architect, I want Python Flask to be responsible only for scraper orchestration and the .NET API to be the sole data API, so that the frontend has a single, consistent data source and race conditions between services are eliminated.

Acceptance Criteria

1. AFTER Phase 2 activation, THE Flask_Service SHALL NOT serve any project data endpoints to the frontend.

2. AFTER Phase 2 activation, THE API SHALL be the exclusive provider of `GET /api/projects` and related project data endpoints to the Angular frontend.

3. AFTER Phase 3 completion, THE Flask_Service SHALL NOT write project data to JSON files in the `scraped_projects/` directory.

4. AFTER Phase 3 completion, THE System SHALL NOT depend on the `scraped_projects/` shared folder for any data exchange between services.

5. WHEN the Admin triggers a scrape via the Admin Panel, THE API SHALL forward the trigger to the Flask_Service and return the resulting `scrape_runs` record to the caller.

Requirement 6: Data Migration Verification

User Story: As a system operator, I want to verify that no data is lost during the migration from JSON files to PostgreSQL, so that I can confidently decommission the JSON file pipeline.

Acceptance Criteria

1. AFTER Phase 1 migration, THE System SHALL ensure that the count of rows in the `projects` table equals the number of valid project directories in `scraped_projects/`.

2. AFTER Phase 1 migration, FOR EACH project directory in `scraped_projects/`, THE System SHALL have a corresponding row in `projects` where `raw_data` contains an equivalent JSON structure to the `view_page_data.json` file.

3. WHEN the `PostgresProjectRepository` returns a project by ID, THE API SHALL produce the same field values as the file-based `ProjectRepository` for the same project identifier.

Requirement 7: User Registration

User Story: As a visitor, I want to create an account with my email and password, so that I can access personalized features like saved properties and favorites.

Acceptance Criteria

1. WHEN a `POST /api/auth/register` request is received with a valid `fullName`, `email`, `password`, and optional `mobile`, THE API SHALL create a new user record in the `users` table with `is_verified = false` and `is_active = true`.

2. THE API SHALL store passwords as BCrypt hashes with cost factor 12 and SHALL NOT store plaintext passwords.

3. WHEN a `POST /api/auth/register` request is received with an `email` that already exists in the `users` table, THE API SHALL return a `409 Conflict` response.

4. WHEN a user is successfully registered, THE API SHALL generate an email verification token, store its SHA-256 hash in `email_verification_tokens` with an expiry of 24 hours, and send a verification email via the Email_Service.

5. IF the Email_Service is unavailable during registration, THEN THE API SHALL still persist the user record and return a `201 Created` response, and SHALL log the email delivery failure.

6. THE API SHALL apply the Input_Sanitizer to `fullName` and `mobile` fields before persisting them.

7. THE `users` table `id` column SHALL be a UUID generated by `gen_random_uuid()`.

Requirement 8: Email Verification

User Story: As a registered user, I want to verify my email address, so that I can access features that require a verified account.

Acceptance Criteria

1. WHEN a `POST /api/auth/verify-email` request is received with a valid, unexpired, unused token, THE API SHALL set `users.is_verified = true` and `users.email_verified_at = NOW()` and mark the token as used.

2. WHEN a `POST /api/auth/verify-email` request is received with an expired token, THE API SHALL return a `400 Bad Request` response with error code `token_expired`.

3. WHEN a `POST /api/auth/verify-email` request is received with an already-used token, THE API SHALL return a `400 Bad Request` response with error code `token_already_used`.

4. WHEN a `POST /api/auth/verify-email` request is received with a token that does not exist, THE API SHALL return a `400 Bad Request` response with error code `invalid_token`.

5. WHEN a `POST /api/auth/resend-verification` request is received with a valid `email`, THE API SHALL generate a new verification token and send a new verification email, invalidating any previous unused tokens for that user.

6. THE API SHALL store only the SHA-256 hash of verification tokens in the database; raw tokens SHALL exist only in transit (email links).

Requirement 9: User Login and Token Issuance

User Story: As a registered user, I want to sign in with my email and password and receive authentication tokens, so that I can make authenticated API requests.

Acceptance Criteria

1. WHEN a `POST /api/auth/login` request is received with a valid email and correct password, THE API SHALL return a `200 OK` response containing an `accessToken` (JWT, 15-minute expiry), a `refreshToken` (opaque UUID, 30-day expiry), an `expiresAt` timestamp, and a `UserProfileDto`.

2. THE API SHALL sign access tokens using HS256 with claims: `sub` (user UUID), `email`, `role`, `name`.

3. THE API SHALL store only the SHA-256 hash of the refresh token in `refresh_tokens`, never the raw value.

4. WHEN a `POST /api/auth/login` request is received with an incorrect password, THE API SHALL return a `401 Unauthorized` response.

5. WHEN a `POST /api/auth/login` request is received with an email that does not exist, THE API SHALL return a `401 Unauthorized` response (same response as wrong password to prevent user enumeration).

6. WHEN a user logs in, THE API SHALL record `users.last_login_at = NOW()`.

7. THE API SHALL enforce IP-based rate limiting of 5 requests per minute on `POST /api/auth/login`.

8. THE API SHALL store `device_info` (browser/device hint) alongside the refresh token record when provided.

Requirement 10: Token Refresh and Logout

User Story: As an authenticated user, I want my session to be automatically extended and to be able to log out securely, so that my session remains valid without requiring frequent re-login while allowing me to revoke access when needed.

Acceptance Criteria

1. WHEN a `POST /api/auth/refresh` request is received with a valid, unexpired, unrevoked refresh token, THE API SHALL issue a new access token and a new refresh token, and SHALL immediately revoke the old refresh token.
2. WHEN a `POST /api/auth/refresh` request is received with an expired refresh token, THE API SHALL return a `401 Unauthorized` response.
3. WHEN a `POST /api/auth/refresh` request is received with a revoked refresh token, THE API SHALL return a `401 Unauthorized` response.
4. WHEN a `POST /api/auth/logout` request is received with a valid user JWT and refresh token, THE API SHALL mark the refresh token as revoked by setting `revoked_at = NOW()`.
5. AFTER logout, THE API SHALL reject subsequent refresh requests using the revoked token.

Requirement 11: Password Reset

User Story: As a user who has forgotten their password, I want to receive a secure reset link by email, so that I can set a new password and regain access to my account.

Acceptance Criteria

1. WHEN a `POST /api/auth/forgot-password` request is received, THE API SHALL return `200 OK` regardless of whether the provided email exists in the system, to prevent email enumeration.
2. WHEN a `POST /api/auth/forgot-password` request is received for a registered email, THE API SHALL generate a cryptographically random 32-byte token, store its SHA-256 hash in `password_reset_tokens` with a 1-hour expiry, and send a password reset email containing the raw token.
3. WHEN a `POST /api/auth/reset-password` request is received with a valid, unexpired, unused token and a new password, THE API SHALL update `users.password_hash` with the BCrypt hash of the new password and mark the reset token as used.

4. WHEN a password is successfully reset, THE API SHALL revoke all existing refresh tokens for that user by setting `revoked_at = NOW()` on each record.

5. WHEN a `POST /api/auth/reset-password` request is received with an expired or already-used token, THE API SHALL return a `400 Bad Request` response with error code `token_invalid`.

6. THE API SHALL store only the SHA-256 hash of password reset tokens in the database; raw tokens SHALL exist only in transit.

Requirement 12: User Profile Management

User Story: As an authenticated user, I want to view and update my profile information, so that I can keep my personal details current.

Acceptance Criteria

1. WHEN a `GET /api/user/profile` request is received with a valid Access_Token, THE API SHALL return the caller's `UserProfileDto` containing `id`, `fullName`, `email`, `mobile`, `avatarUrl`, `isVerified`, and `createdAt`.

2. WHEN a `PUT /api/user/profile` request is received with a valid Access_Token and updated `fullName`, `mobile`, or `avatarUrl`, THE API SHALL persist the changes to the `users` table and update `updated_at`.

3. WHEN a `PUT /api/user/change-password` request is received with a valid Access_Token, correct `currentPassword`, and a valid `newPassword`, THE API SHALL update `users.password_hash` with the BCrypt hash of the new password.

4. IF the `currentPassword` provided in a `PUT /api/user/change-password` request does not match the stored hash, THEN THE API SHALL return a `400 Bad Request` response with error code `incorrect_current_password`.

5. WHEN a `DELETE /api/user/account` request is received with a valid Access_Token, THE API SHALL soft-delete the account by setting `is_active = false` and SHALL revoke all refresh tokens for that user.

6. THE API SHALL apply the Input_Sanitizer to `fullName` and `mobile` before persisting updates.

7. WHILE a user's `is_active` is `false`, THE API SHALL return `401 Unauthorized` for all authenticated requests from that user.

Requirement 13: Saved Properties

User Story: As an authenticated user, I want to bookmark projects I want to revisit, so that I can quickly find and review properties I am seriously considering.

Acceptance Criteria

1. WHEN a `POST /api/user/saved-properties` request is received with a valid Access_Token and a `projectId` that exists in the `projects` table, THE API SHALL insert a row into `saved_properties` and return the created `SavedPropertyDto`.

2. WHEN a `POST /api/user/saved-properties` request is received with a `projectId` that is already saved by the same user, THE API SHALL return a `409 Conflict` response.

3. WHEN a `GET /api/user/saved-properties` request is received with a valid Access_Token, THE API SHALL return all saved properties for the authenticated user, including `projectName`, `locality`, `district`, `projectStatus`, `notes`, and `savedAt`.

4. WHEN a `DELETE /api/user/saved-properties/{projectId}` request is received with a valid Access_Token, THE API SHALL remove the matching row from `saved_properties` for the authenticated user.

5. WHEN a `GET /api/user/saved-properties/{projectId}/exists` request is received with a valid Access_Token, THE API SHALL return a boolean indicating whether the project is currently saved by the authenticated user.

6. IF a `POST /api/user/saved-properties` request is received with a `projectId` that does not exist in the `projects` table, THEN THE API SHALL return a `404 Not Found` response.

7. THE System SHALL enforce the UNIQUE constraint on `(user_id, project_id)` in `saved_properties` at the database level.

8. WHEN a project is deleted from the `projects` table, THE System SHALL cascade-delete all corresponding rows in `saved_properties`.

Requirement 14: Favorites

User Story: As an authenticated user, I want to quickly mark projects as favorites using a heart button, so that I can maintain a lightweight collection of interesting properties separate from my deliberate bookmarks.

Acceptance Criteria

1. WHEN a `POST /api/user/favorites` request is received with a valid Access_Token and a `projectId` that exists in the `projects` table, THE API SHALL insert a row into `favorites` and return `201 Created`.

2. WHEN a `POST /api/user/favorites` request is received with a `projectId` that is already favorited by the same user, THE API SHALL return a `409 Conflict` response.

3. WHEN a `GET /api/user/favorites` request is received with a valid Access_Token, THE API SHALL return all favorited projects for the authenticated user, ordered by `created_at` descending.

4. WHEN a `DELETE /api/user/favorites/{projectId}` request is received with a valid Access_Token, THE API SHALL remove the matching row from `favorites` for the authenticated user.

5. WHEN a `GET /api/user/favorites/{projectId}/exists` request is received with a valid Access_Token, THE API SHALL return a boolean indicating whether the project is currently favorited by the authenticated user.

6. IF a `POST /api/user/favorites` request is received with a `projectId` that does not exist in the `projects` table, THEN THE API SHALL return a `404 Not Found` response.

7. THE System SHALL enforce the UNIQUE constraint on `(user_id, project_id)` in `favorites` at the database level.

8. WHEN a project is deleted from the `projects` table, THE System SHALL cascade-delete all corresponding rows in `favorites`.

Requirement 15: Saved Searches

User Story: As an authenticated user, I want to save my search filter criteria under a custom name and re-run them at any time, so that I can quickly re-access property searches I perform regularly.

Acceptance Criteria

1. WHEN a `POST /api/user/saved-searches` request is received with a valid Access_Token, a non-empty `name`, and a non-empty `filters` object, THE API SHALL insert a row into `saved_searches` and return the created `SavedSearchDto`.

2. WHEN a `GET /api/user/saved-searches` request is received with a valid Access_Token, THE API SHALL return all saved searches for the authenticated user, including `id`, `name`, `filters`, `resultCount`, `lastRunAt`, and `createdAt`.

3. WHEN a `PUT /api/user/saved-searches/{id}` request is received with a valid Access_Token and the search belongs to the authenticated user, THE API SHALL update the `name` and/or `filters` and set `updated_at = NOW()`.

4. WHEN a `DELETE /api/user/saved-searches/{id}` request is received with a valid Access_Token and the search belongs to the authenticated user, THE API SHALL remove the row from `saved_searches`.

5. WHEN a `POST /api/user/saved-searches/{id}/run` request is received with a valid Access_Token, THE API SHALL execute the saved filter criteria against the `projects` table, update `last_run_at` and `result_count` on the saved search record, and return the matching projects.

6. IF a `PUT` or `DELETE` or `POST /run` request targets a saved search that does not belong to the authenticated user, THEN THE API SHALL return a `403 Forbidden` response.

7. IF a `PUT` or `DELETE` or `POST /run` request targets a saved search `id` that does not exist, THEN THE API SHALL return a `404 Not Found` response.

8. THE `saved_searches.filters` column SHALL be stored as JSONB and SHALL support at minimum the fields: `pinCodes`, `district`, `minFlats`, `maxFlats`, `projectStatus`, and `sortBy`.

Requirement 16: Comparison Results

User Story: As an authenticated user, I want to save the results of project comparisons, so that I can return to a comparison later without having to recreate it.

Acceptance Criteria

1. WHEN a `POST /api/user/comparisons` request is received with a valid Access_Token, a `projectIds` array of at least 2 valid project IDs, and an optional `name`, THE API SHALL insert a row into `comparison_results` and return the created `ComparisonResultDto`.

2. WHEN a `GET /api/user/comparisons` request is received with a valid Access_Token, THE API SHALL return all saved comparisons for the authenticated user, including `id`, `name`, `projectIds`, and `createdAt`.

3. WHEN a `GET /api/user/comparisons/{id}` request is received with a valid Access_Token and the comparison belongs to the authenticated user, THE API SHALL return the full comparison record including the `snapshot` JSONB field if present.

4. WHEN a `DELETE /api/user/comparisons/{id}` request is received with a valid Access_Token and the comparison belongs to the authenticated user, THE API SHALL remove the row from `comparison_results`.

5. IF a `GET`, `DELETE` request targets a comparison that does not belong to the authenticated user, THEN THE API SHALL return a `403 Forbidden` response.

6. IF a `GET`, `DELETE` request targets a comparison `id` that does not exist, THEN THE API SHALL return a `404 Not Found` response.

7. IF a `POST /api/user/comparisons` request contains a `projectId` that does not exist in the `projects` table, THEN THE API SHALL return a `422 Unprocessable Entity` response listing the invalid IDs.

Requirement 17: Authentication and Authorization Guards

User Story: As a system operator, I want all user-specific endpoints to require a valid JWT, so that user data is accessible only to its owner and unauthorized access is prevented.

Acceptance Criteria

1. WHEN a request to any `/api/user/*` endpoint is received without an `Authorization: Bearer <token>` header, THE API SHALL return `401 Unauthorized`.

2. WHEN a request to any `/api/user/*` endpoint is received with an expired Access_Token, THE API SHALL return `401 Unauthorized` with error code `token_expired`.

3. WHEN a request to any `/api/user/*` endpoint is received with a malformed or invalid Access_Token, THE API SHALL return `401 Unauthorized` with error code `invalid_token`.

4. WHILE a user's `is_verified` is `false`, THE API SHALL return `403 Forbidden` with error code `email_not_verified` for requests to endpoints that require a verified account (saved-properties, favorites, saved-searches, comparisons).

5. THE API SHALL ensure that a user can only access, modify, or delete their own saved properties, favorites, saved searches, and comparison results by validating `user_id` against the authenticated user's `sub` claim.

Requirement 18: Security Controls

User Story: As a security engineer, I want all sensitive data and authentication mechanisms to follow security best practices, so that user accounts and data are protected from common attack vectors.

Acceptance Criteria

1. THE API SHALL hash all passwords using BCrypt with a cost factor of 12 before storing them.

2. THE API SHALL store only SHA-256 hashes of refresh tokens, password reset tokens, and email verification tokens in the database; raw token values SHALL NOT be persisted.

3. THE API SHALL enforce IP-based rate limiting of 5 requests per minute on the `POST /api/auth/login` endpoint.

4. THE API SHALL always return `200 OK` from `POST /api/auth/forgot-password` regardless of whether the email address exists, to prevent email enumeration.

5. THE API SHALL use parameterized queries via Dapper for all SQL operations to prevent SQL injection.

6. THE API SHALL apply the existing Input_Sanitizer to all user-provided string inputs before persisting them.

7. WHEN a password reset is completed, THE API SHALL revoke all active refresh tokens for the affected user, forcing re-authentication on all devices.

8. THE API SHALL sign JWTs using HS256, with a documented upgrade path to RS256 asymmetric keys for production deployment.